

Importing libraries

```
In [ ]: import nbconvert
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import missingno as mso
import seaborn as sns
import warnings
import os
import scipy

from scipy import stats
from scipy.stats import pearsonr
from scipy.stats import ttest_ind
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE

from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import CategoricalNB
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
from xgboost import XGBClassifier
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
```

Reading dataset

```
In [2]: df = pd.read_csv("loan-eligible.csv")
df.head()
```

Out[2]:

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Credit_History
0	LP001002	Male	No	0	Graduate	No	5849	1
1	LP001003	Male	Yes	1	Graduate	No	4583	1
2	LP001005	Male	Yes	0	Graduate	Yes	3000	1
3	LP001006	Male	Yes	0	Not Graduate	No	2583	1
4	LP001008	Male	No	0	Graduate	No	6000	1

In [3]: `print(df.shape)`

(614, 13)

Data exploration

- Loan ID

In [4]: `df.Loan_ID.value_counts(dropna=False)`

Out[4]:

Loan_ID	count
LP002990	1
LP001002	1
LP001003	1
LP001005	1
LP001006	1
...	...
LP001024	1
LP001020	1
LP001018	1
LP001014	1
LP001013	1

614 rows × 1 columns

dtype: int64

- Gender

```
In [6]: df.Gender.value_counts(dropna=False)
```

```
Out[6]:
```

	count
Gender	
Male	489
Female	112
NaN	13

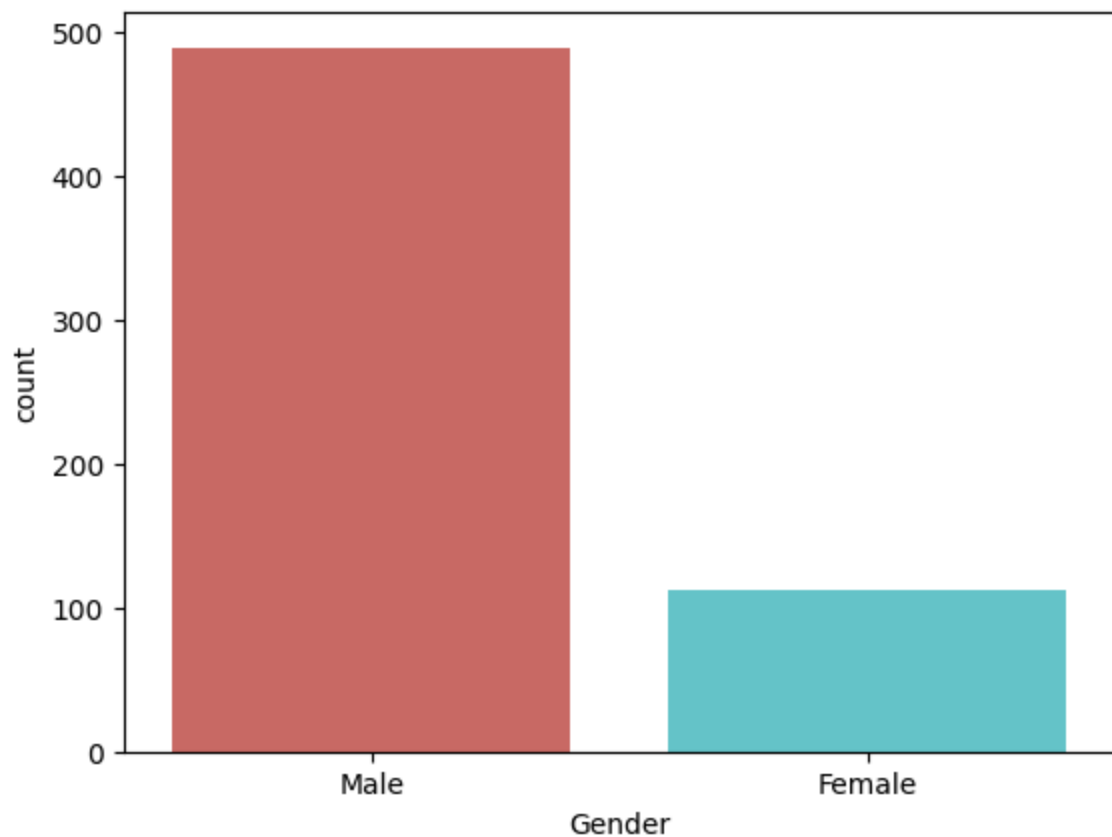
dtype: int64

```
In [7]: sns.countplot(x="Gender", data=df, palette="hls")  
plt.show()
```

/tmp/ipython-input-2604761689.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x="Gender", data=df, palette="hls")
```



```
In [8]: countMale = len(df[df.Gender == 'Male'])  
countFemale = len(df[df.Gender == 'Female'])
```

```
countNull = len(df[df.Gender.isnull()])

print("Percentage of Male applicant: {:.2f}%".format((countMale / (len(df.Gender))*
print("Percentage of Female applicant: {:.2f}%".format((countFemale / (len(df.Gende
print("Missing values percentage: {:.2f}%".format((countNull / (len(df.Gender))*100
```

Percentage of Male applicant: 79.64%
 Percentage of Female applicant: 18.24%
 Missing values percentage: 2.12%

- Married

```
In [9]: df.Married.value_counts(dropna=False)
```

```
Out[9]:
```

	count
Married	
Yes	398
No	213
NaN	3

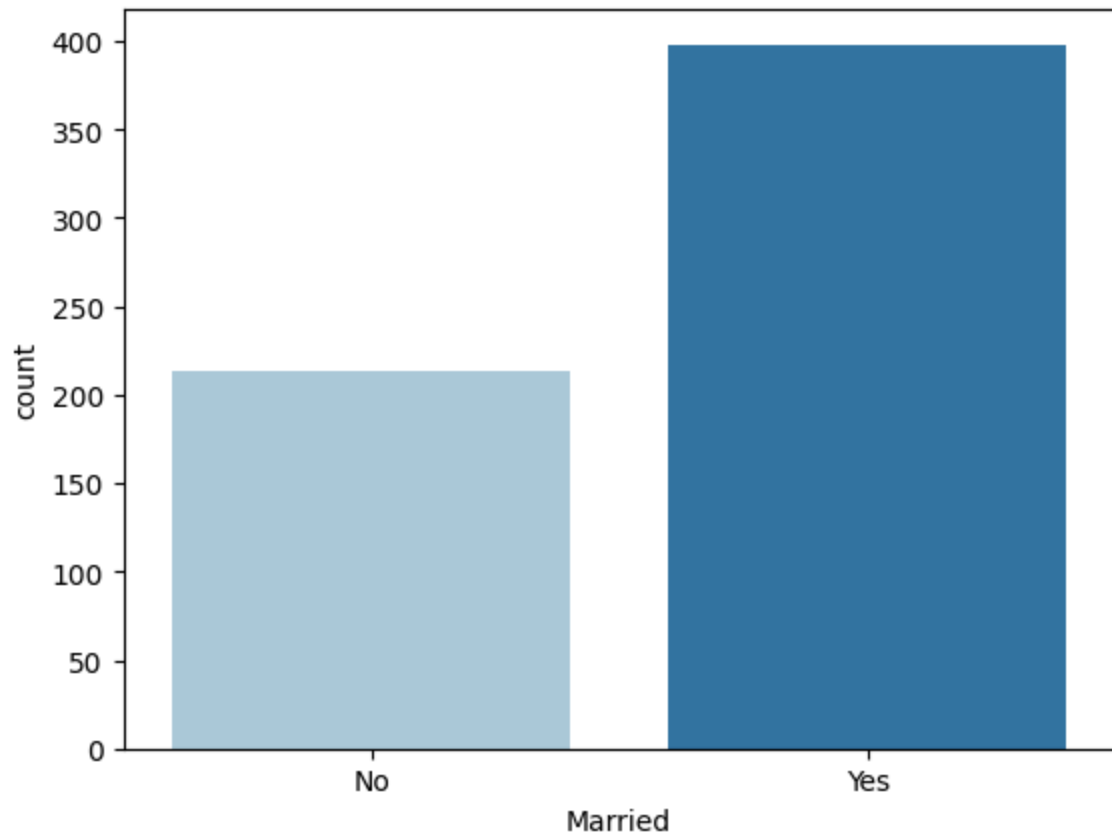
dtype: int64

```
In [10]: sns.countplot(x="Married", data=df, palette="Paired")
plt.show()
```

/tmp/ipython-input-3626669607.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x="Married", data=df, palette="Paired")
```



```
In [11]: countMarried = len(df[df.Married == 'Yes'])
countNotMarried = len(df[df.Married == 'No'])
countNull = len(df[df.Married.isnull()])

print("Percentage of married: {:.2f}%".format((countMarried / (len(df.Married))*100
print("Percentage of Not married applicant: {:.2f}%".format((countNotMarried / (len
print("Missing values percentage: {:.2f}%".format((countNull / (len(df.Married))*10
```

Percentage of married: 64.82%
 Percentage of Not married applicant: 34.69%
 Missing values percentage: 0.49%

- Education

```
In [12]: df.Education.value_counts(dropna=False)
```

Out[12]:

Education		count
Graduate		480
Not Graduate		134

dtype: int64

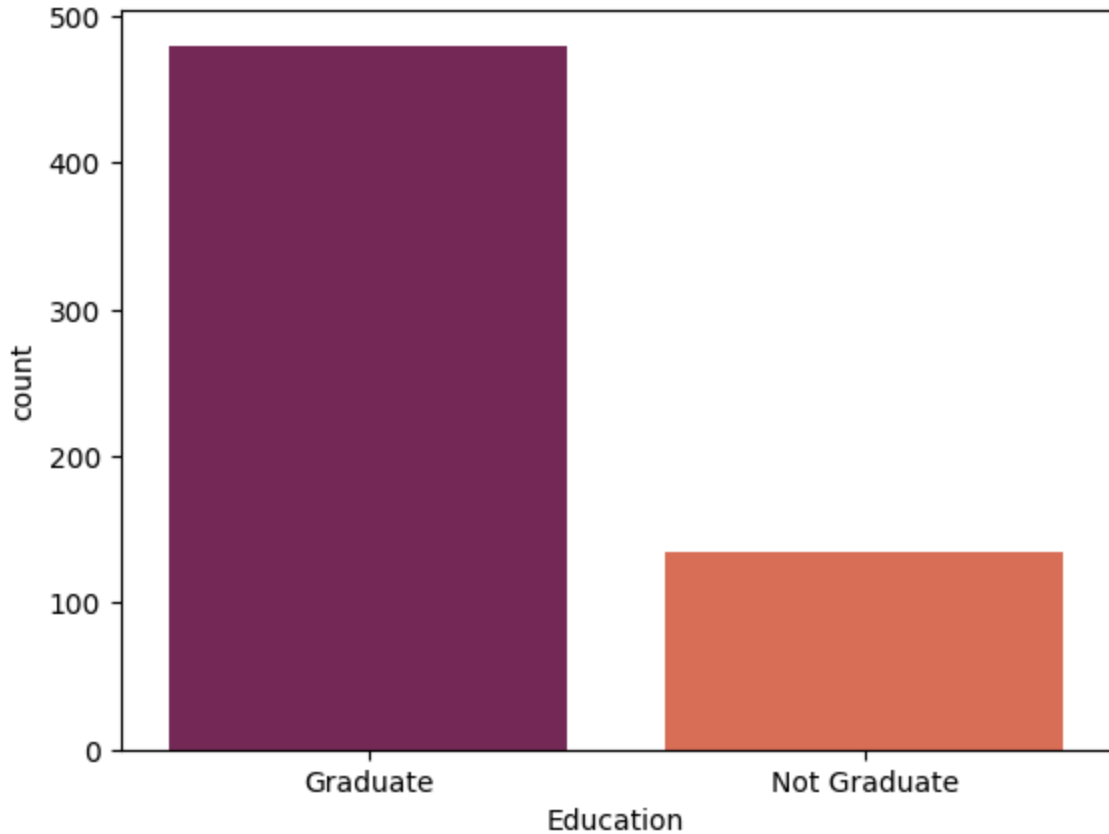
```
In [13]: sns.countplot(x="Education", data=df, palette="rocket")
```

```
plt.show()
```

/tmp/ipython-input-4023769793.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x="Education", data=df, palette="rocket")
```



```
In [14]: countGraduate = len(df[df.Education == 'Graduate'])
countNotGraduate = len(df[df.Education == 'Not Graduate'])
countNull = len(df[df.Education.isnull()])

print("Percentage of graduate applicant: {:.2f}%".format((countGraduate / (len(df.Education))))
print("Percentage of Not graduate applicant: {:.2f}%".format((countNotGraduate / (len(df.Education))))
print("Missing values percentage: {:.2f}%".format((countNull / (len(df.Education))))
```

```
Percentage of graduate applicant: 78.18%
Percentage of Not graduate applicant: 21.82%
Missing values percentage: 0.00%
```

- Self-Employed

```
In [15]: df.Self_Employed.value_counts(dropna=False)
```

Out[15]:

count	
Self_Employed	
No	500
Yes	82
NaN	32

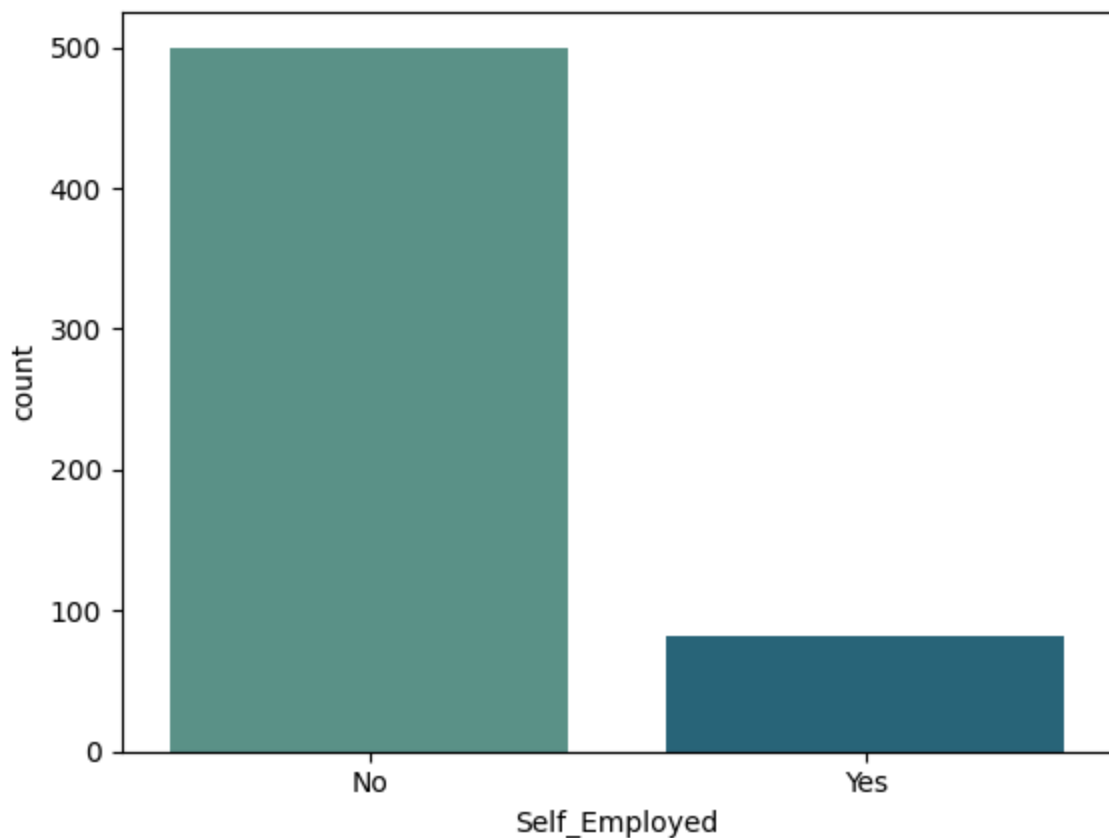
dtype: int64

```
In [16]: sns.countplot(x="Self_Employed", data=df, palette="crest")
plt.show()
```

/tmp/ipython-input-1963651902.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x="Self_Employed", data=df, palette="crest")
```



```
In [17]: countNo = len(df[df.Self_Employed == 'No'])
countYes = len(df[df.Self_Employed == 'Yes'])
countNull = len(df[df.Self_Employed.isnull()])

print("Percentage of Not self employed: {:.2f}%".format((countNo / (len(df.Self_Emp
print("Percentage of self employed: {:.2f}%".format((countYes / (len(df.Self_Employ
print("Missing values percentage: {:.2f}%".format((countNull / (len(df.Self_Employe
```

Percentage of Not self employed: 81.43%

Percentage of self employed: 13.36%

Missing values percentage: 5.21%

- Credit history

```
In [18]: df.Credit_History.value_counts(dropna=False)
```

```
Out[18]:
```

	count
Credit_History	
1.0	475
0.0	89
NaN	50

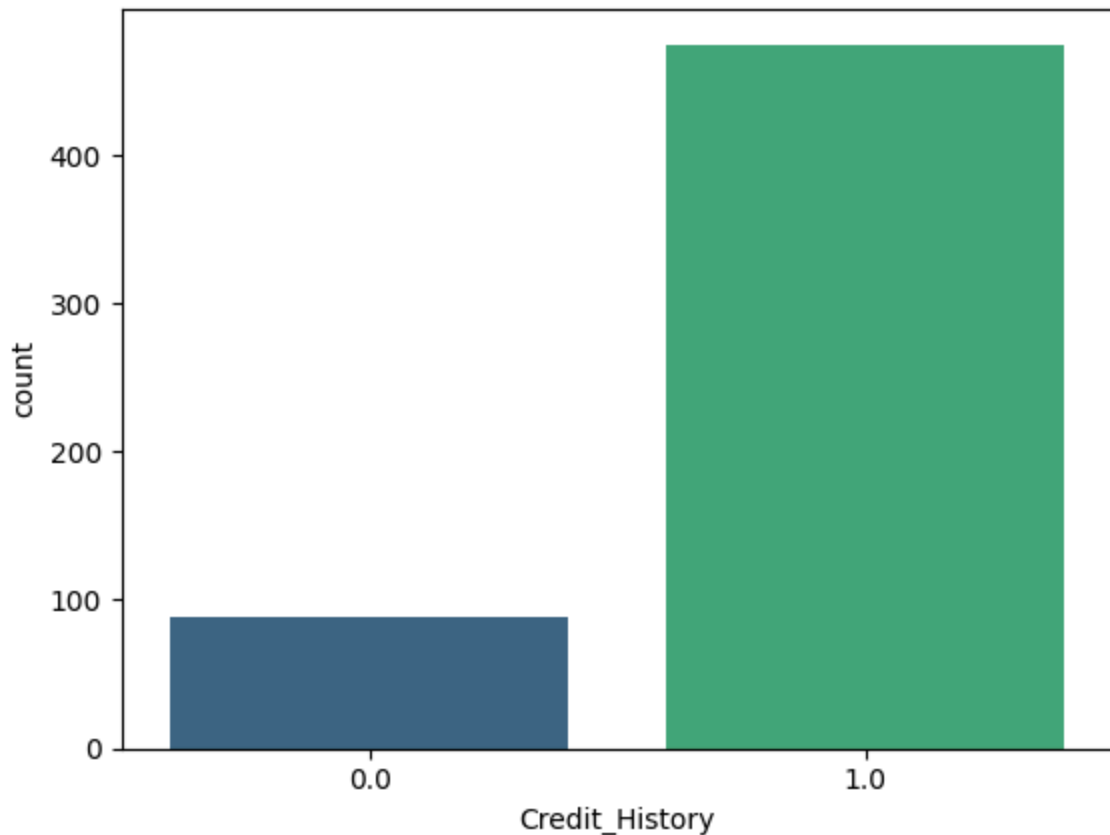
dtype: int64

```
In [19]: sns.countplot(x="Credit_History", data=df, palette="viridis")
plt.show()
```

/tmp/ipython-input-261100619.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x="Credit_History", data=df, palette="viridis")
```

```
In [20]: count1 = len(df[df.Credit_History == 1])
count0 = len(df[df.Credit_History == 0])
countNull = len(df[df.Credit_History.isnull()])

print("Percentage of Good credit history: {:.2f}%".format((count1 / (len(df.Credit_
print("Percentage of Bad credit history: {:.2f}%".format((count0 / (len(df.Credit_H
print("Missing values percentage: {:.2f}%".format((countNull / (len(df.Credit_Histo
```

Percentage of Good credit history: 77.36%
 Percentage of Bad credit history: 14.50%
 Missing values percentage: 8.14%

- Property area

```
In [21]: df.Property_Area.value_counts(dropna=False)
```

Out[21]:

	count
Property_Area	
Semiurban	233
Urban	202
Rural	179

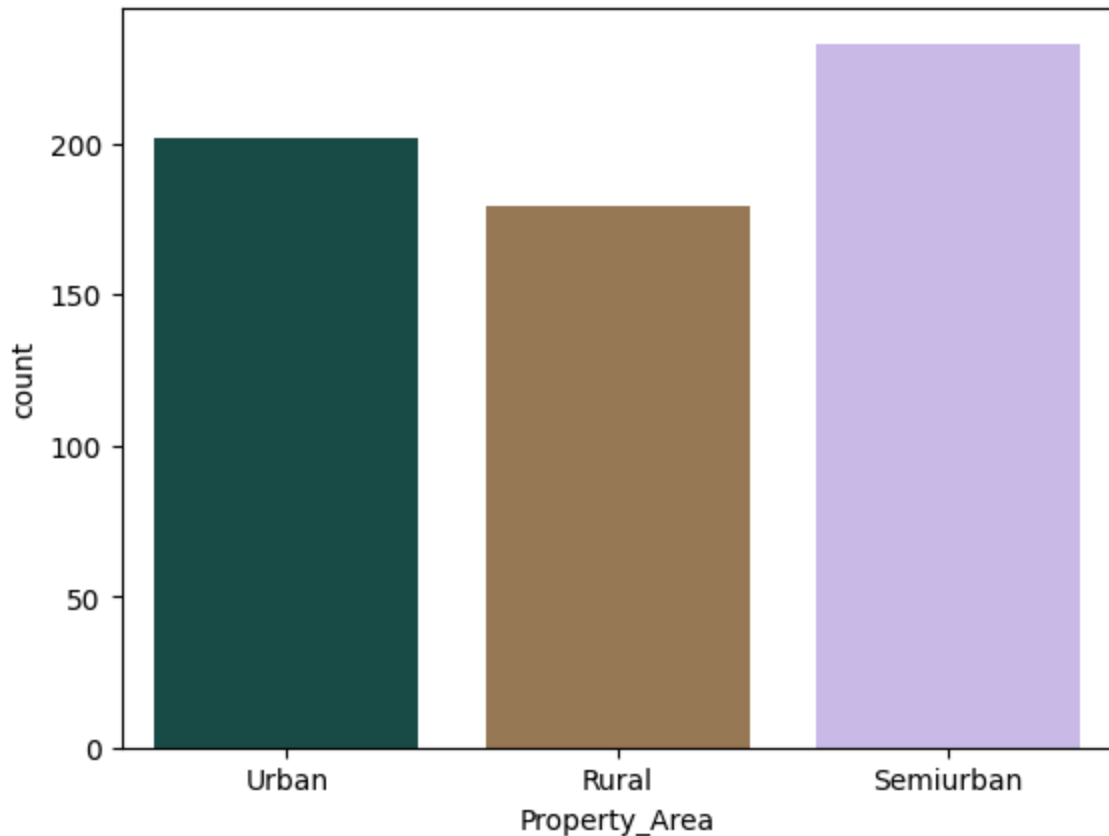
dtype: int64

```
In [22]: sns.countplot(x="Property_Area", data=df, palette="cubehelix")
plt.show()
```

/tmp/ipython-input-1765907190.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x="Property_Area", data=df, palette="cubehelix")
```



```
In [23]: countUrban = len(df[df.Property_Area == 'Urban'])
countRural = len(df[df.Property_Area == 'Rural'])
countSemiurban = len(df[df.Property_Area == 'Semiurban'])
countNull = len(df[df.Property_Area.isnull()])

print("Percentage of Urban: {:.2f}%".format((countUrban / (len(df.Property_Area))*100))
print("Percentage of Rural: {:.2f}%".format((countRural / (len(df.Property_Area))*100))
print("Percentage of Semiurban: {:.2f}%".format((countSemiurban / (len(df.Property_Area))*100))
print("Missing values percentage: {:.2f}%".format((countNull / (len(df.Property_Area))*100))
```

Percentage of Urban: 32.90%

Percentage of Rural: 29.15%

Percentage of Semiurban: 37.95%

Missing values percentage: 0.00%

- Loan status

```
In [24]: df.Loan_Status.value_counts(dropna=False)
```

Out[24]:

count	
Loan_Status	
Y	422
N	192

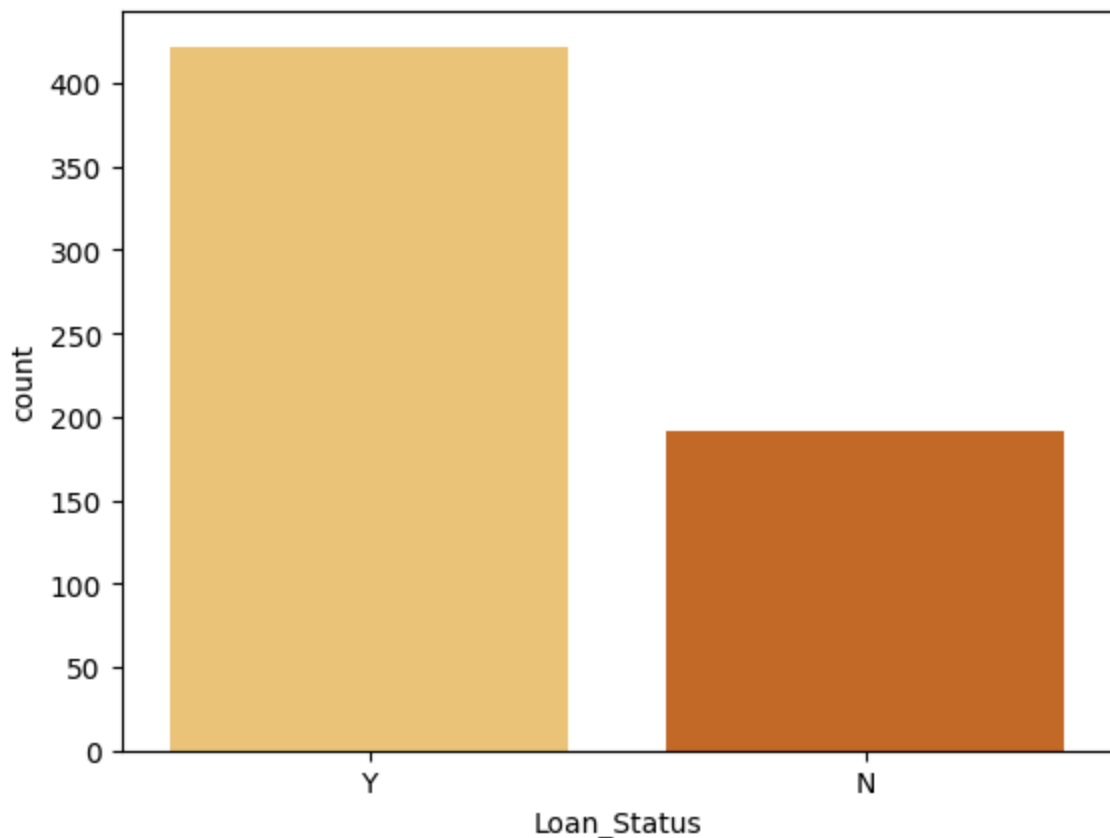
dtype: int64

```
In [25]: sns.countplot(x="Loan_Status", data=df, palette="YlOrBr")
plt.show()
```

/tmp/ipython-input-1110804027.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x="Loan_Status", data=df, palette="YlOrBr")
```



```
In [26]: countY = len(df[df.Loan_Status == 'Y'])
countN = len(df[df.Loan_Status == 'N'])
countNull = len(df[df.Loan_Status.isnull()])

print("Percentage of Approved: {:.2f}%".format((countY / (len(df.Loan_Status))*100))
print("Percentage of Rejected: {:.2f}%".format((countN / (len(df.Loan_Status))*100))
print("Missing values percentage: {:.2f}%".format((countNull / (len(df.Loan_Status))
```

Percentage of Approved: 68.73%
Percentage of Rejected: 31.27%
Missing values percentage: 0.00%

- Loan amount term

```
In [27]: df.Loan_Amount_Term.value_counts(dropna=False)
```

```
Out[27]:
```

Loan_Amount_Term	count
360.0	512
180.0	44
480.0	15
NaN	14
300.0	13
84.0	4
240.0	4
120.0	3
60.0	2
36.0	2
12.0	1

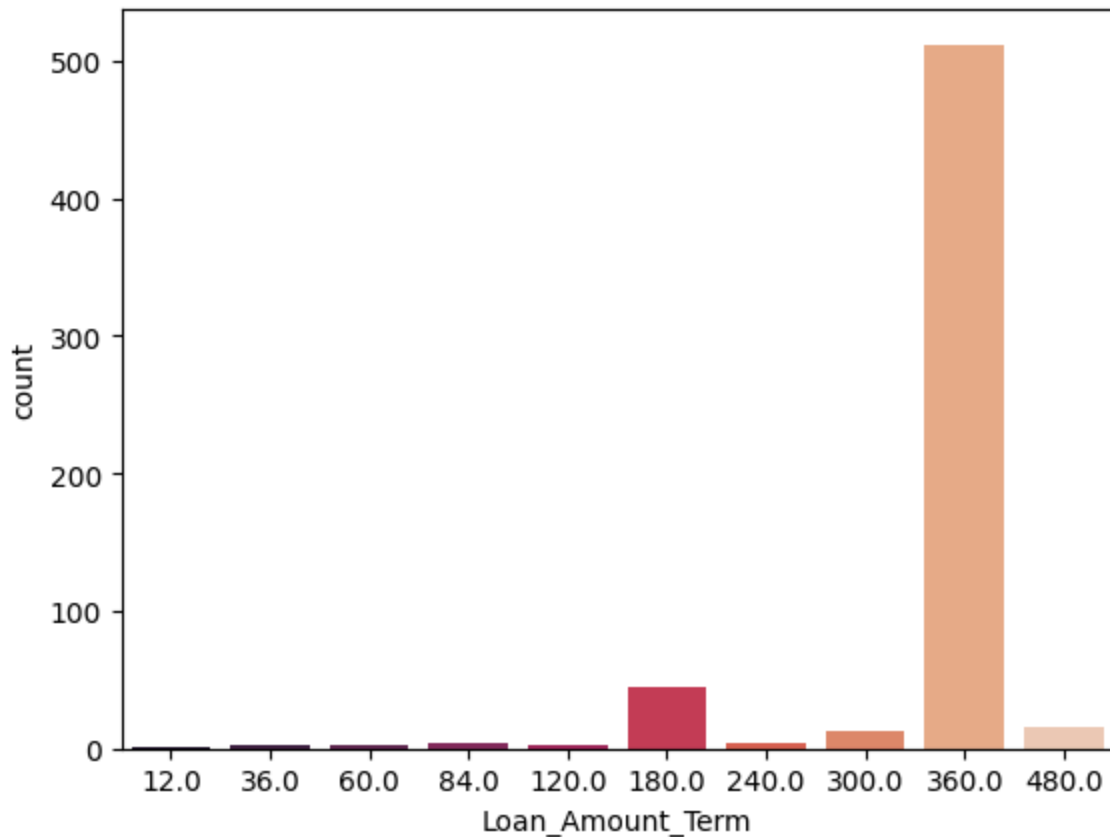
dtype: int64

```
In [28]: sns.countplot(x="Loan_Amount_Term", data=df, palette="rocket")  
plt.show()
```

/tmp/ipython-input-32458805.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x="Loan_Amount_Term", data=df, palette="rocket")
```



```
In [29]: count12 = len(df[df.Loan_Amount_Term == 12.0])
count36 = len(df[df.Loan_Amount_Term == 36.0])
count60 = len(df[df.Loan_Amount_Term == 60.0])
count84 = len(df[df.Loan_Amount_Term == 84.0])
count120 = len(df[df.Loan_Amount_Term == 120.0])
count180 = len(df[df.Loan_Amount_Term == 180.0])
count240 = len(df[df.Loan_Amount_Term == 240.0])
count300 = len(df[df.Loan_Amount_Term == 300.0])
count360 = len(df[df.Loan_Amount_Term == 360.0])
count480 = len(df[df.Loan_Amount_Term == 480.0])
countNull = len(df[df.Loan_Amount_Term.isnull()])

print("Percentage of 12: {:.2f}%".format((count12 / (len(df.Loan_Amount_Term))*100))
print("Percentage of 36: {:.2f}%".format((count36 / (len(df.Loan_Amount_Term))*100))
print("Percentage of 60: {:.2f}%".format((count60 / (len(df.Loan_Amount_Term))*100))
print("Percentage of 84: {:.2f}%".format((count84 / (len(df.Loan_Amount_Term))*100))
print("Percentage of 120: {:.2f}%".format((count120 / (len(df.Loan_Amount_Term))*100))
print("Percentage of 180: {:.2f}%".format((count180 / (len(df.Loan_Amount_Term))*100))
print("Percentage of 240: {:.2f}%".format((count240 / (len(df.Loan_Amount_Term))*100))
print("Percentage of 300: {:.2f}%".format((count300 / (len(df.Loan_Amount_Term))*100))
print("Percentage of 360: {:.2f}%".format((count360 / (len(df.Loan_Amount_Term))*100))
print("Percentage of 480: {:.2f}%".format((count480 / (len(df.Loan_Amount_Term))*100))
print("Missing values percentage: {:.2f}%".format((countNull / (len(df.Loan_Amount_
```

Percentage of 12: 0.16%
Percentage of 36: 0.33%
Percentage of 60: 0.33%
Percentage of 84: 0.65%
Percentage of 120: 0.49%
Percentage of 180: 7.17%
Percentage of 240: 0.65%
Percentage of 300: 2.12%
Percentage of 360: 83.39%
Percentage of 480: 2.44%
Missing values percentage: 2.28%

- Numerical variables

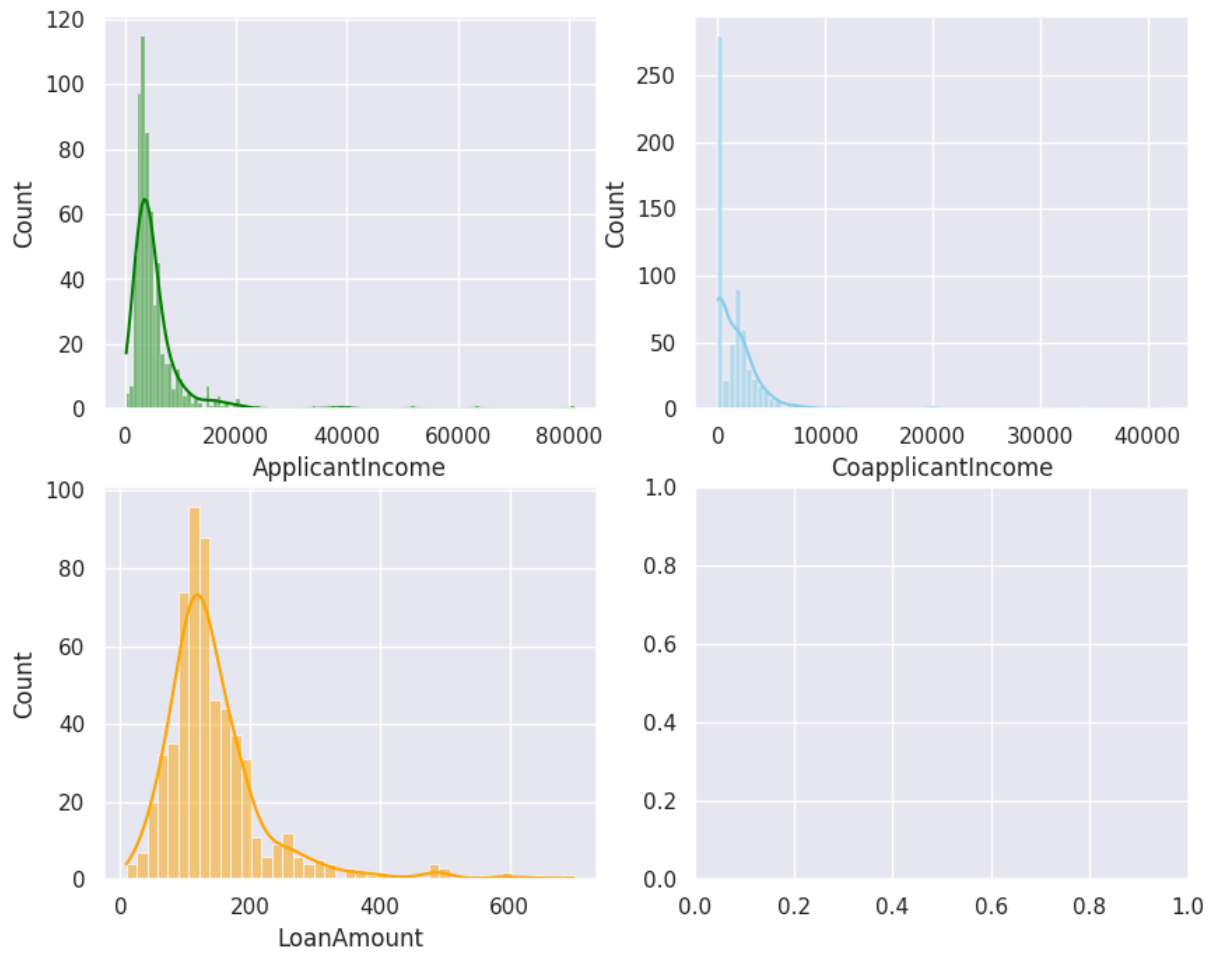
```
In [30]: df[['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount']].describe()
```

```
Out[30]:
```

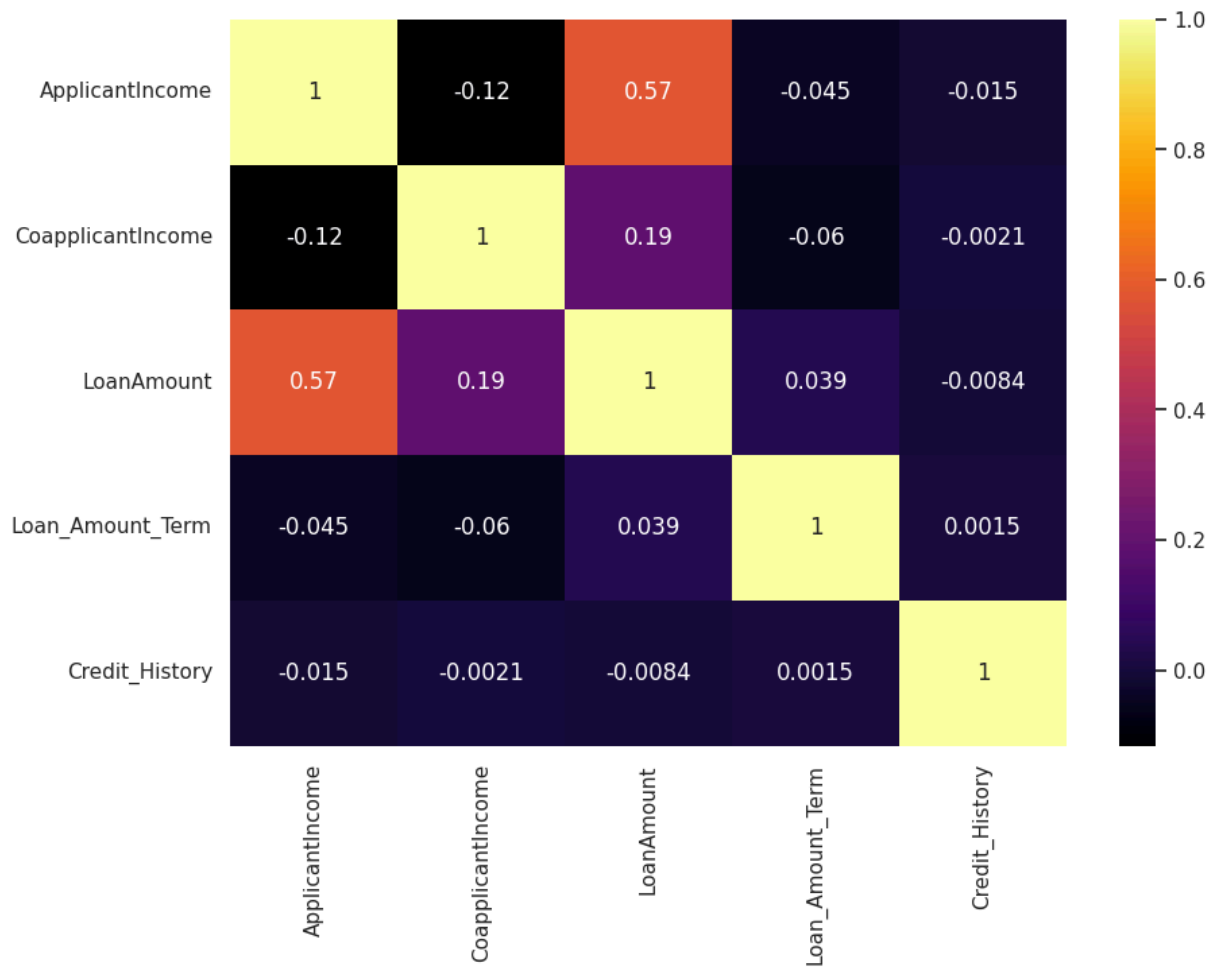
	ApplicantIncome	CoapplicantIncome	LoanAmount
count	614.000000	614.000000	592.000000
mean	5403.459283	1621.245798	146.412162
std	6109.041673	2926.248369	85.587325
min	150.000000	0.000000	9.000000
25%	2877.500000	0.000000	100.000000
50%	3812.500000	1188.500000	128.000000
75%	5795.000000	2297.250000	168.000000
max	81000.000000	41667.000000	700.000000

```
In [31]: sns.set(style="darkgrid")
fig, axs = plt.subplots(2, 2, figsize=(10, 8))

sns.histplot(data=df, x="ApplicantIncome", kde=True, ax=axs[0, 0], color='green')
sns.histplot(data=df, x="CoapplicantIncome", kde=True, ax=axs[0, 1], color='skyblue')
sns.histplot(data=df, x="LoanAmount", kde=True, ax=axs[1, 0], color='orange');
```



```
In [32]: plt.figure(figsize=(10,7))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='inferno');
```

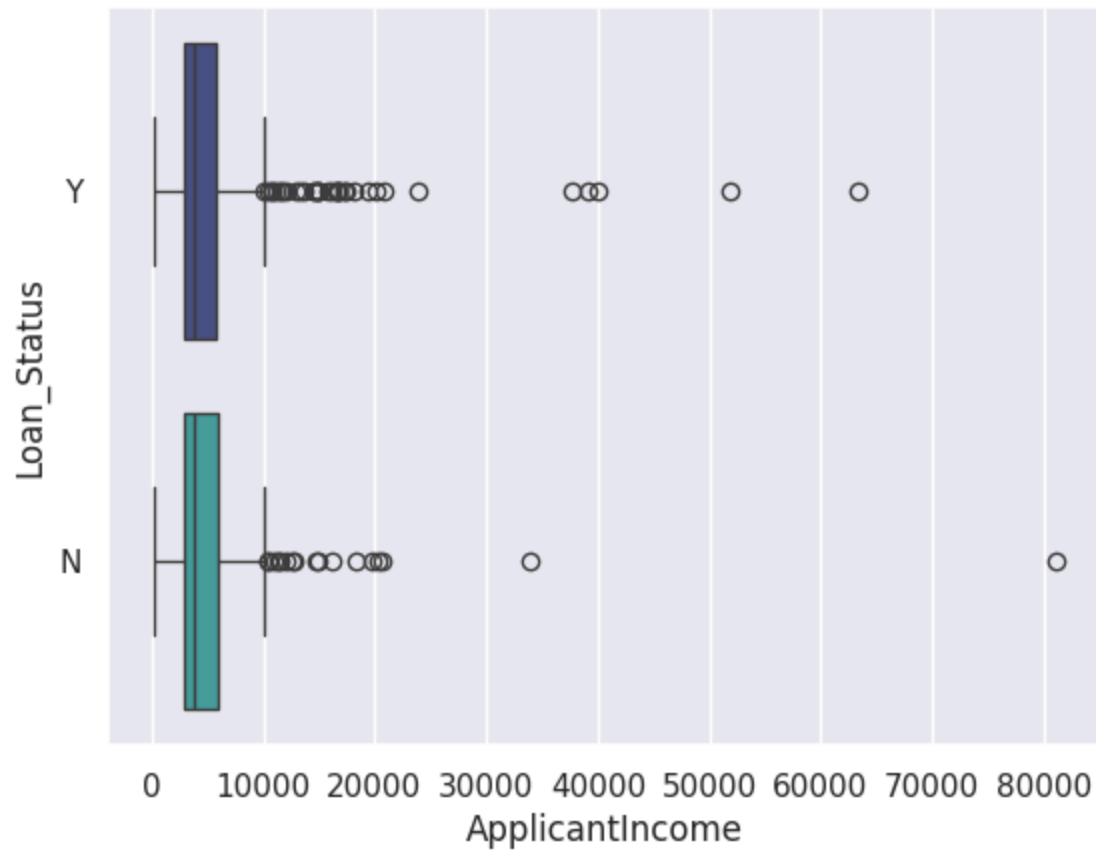


```
In [33]: sns.boxplot(x="ApplicantIncome", y="Loan_Status", data=df, palette="mako");
```

/tmp/ipython-input-3744591124.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(x="ApplicantIncome", y="Loan_Status", data=df, palette="mako");
```

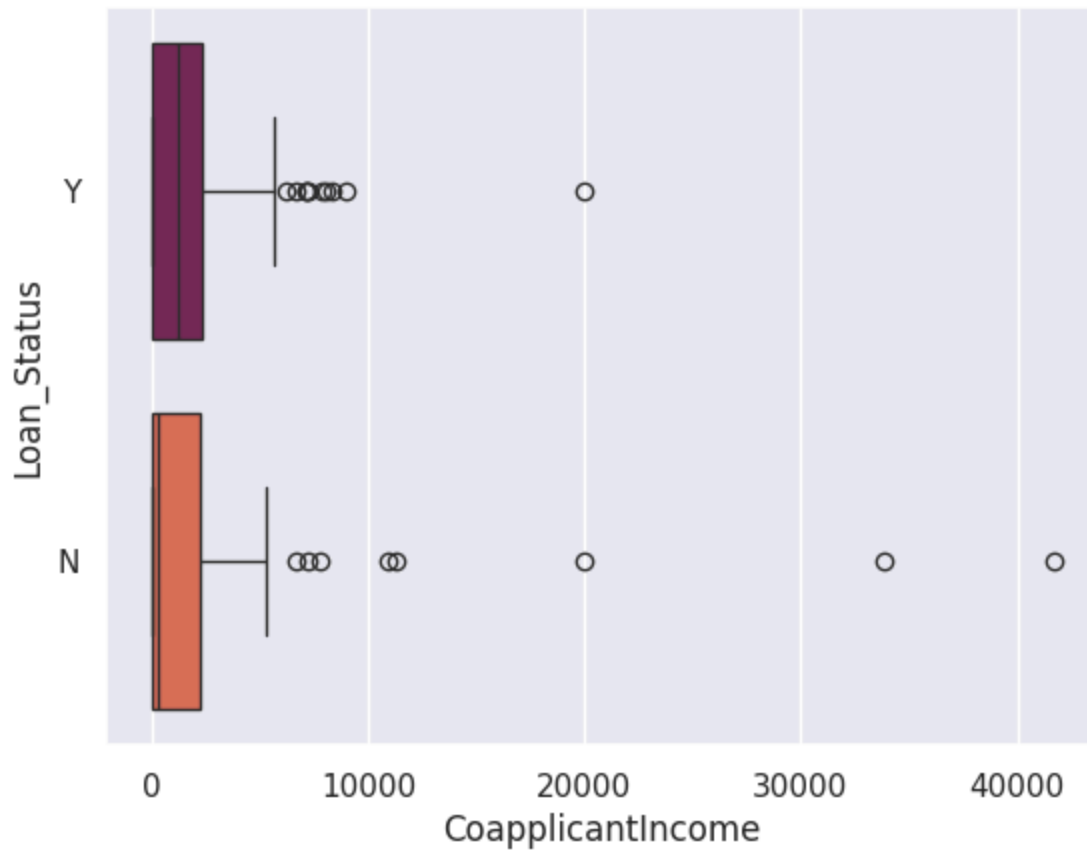



```
In [34]: sns.boxplot(x="CoapplicantIncome", y="Loan_Status", data=df, palette="rocket");
```

/tmp/ipython-input-3243319138.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(x="CoapplicantIncome", y="Loan_Status", data=df, palette="rocket");
```

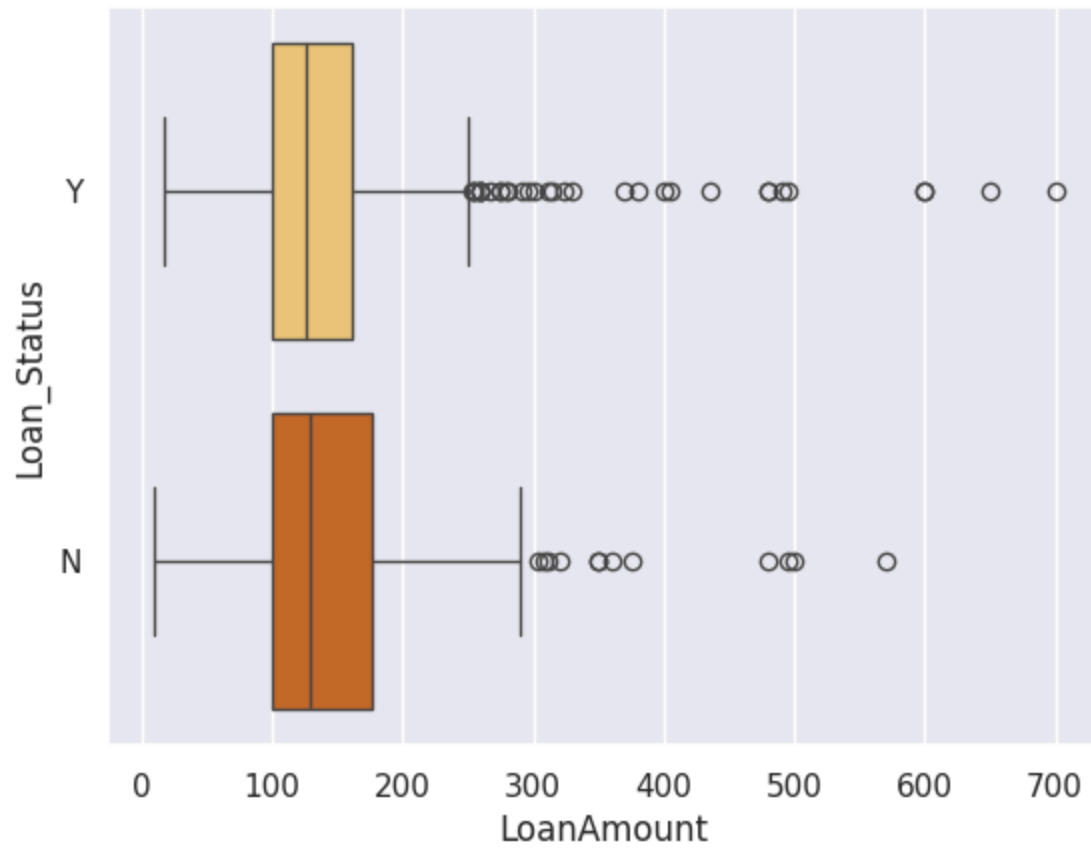


```
In [35]: sns.boxplot(x="LoanAmount", y="Loan_Status", data=df, palette="YlOrBr");
```

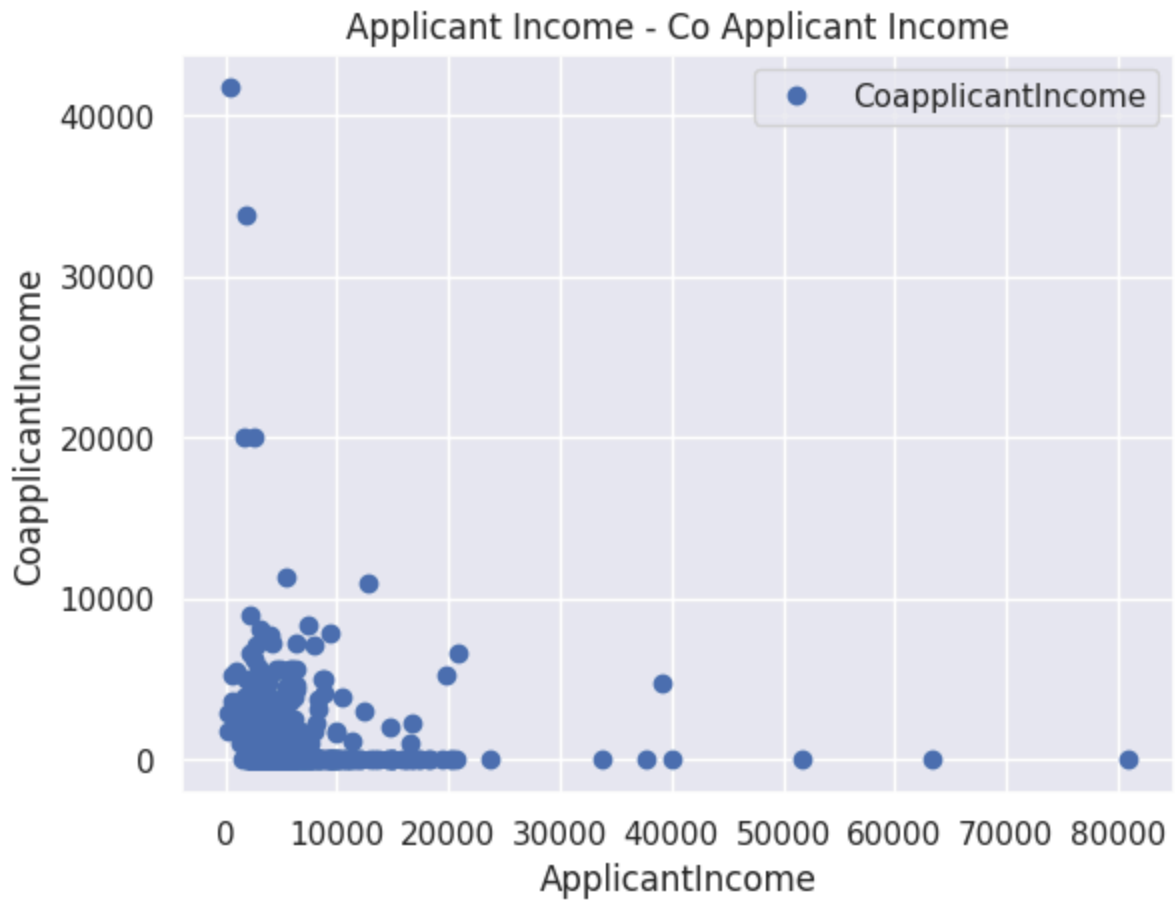
/tmp/ipython-input-3133398061.py:1: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.boxplot(x="LoanAmount", y="Loan_Status", data=df, palette="YlOrBr");
```



```
In [36]: df.plot(x='ApplicantIncome', y='CoapplicantIncome', style='o')
plt.title('Applicant Income - Co Applicant Income')
plt.xlabel('ApplicantIncome')
plt.ylabel('CoapplicantIncome')
plt.show()
print('Pearson correlation:', df['ApplicantIncome'].corr(df['CoapplicantIncome']))
print('T Test and P value: \n', stats.ttest_ind(df['ApplicantIncome'], df['CoapplicantIncome']))
```



Pearson correlation: -0.11660458122889966

T Test and P value:

```
TtestResult(statistic=np.float64(13.835753259915665), pvalue=np.float64(1.460983948423972e-40), df=np.float64(1226.0))
```

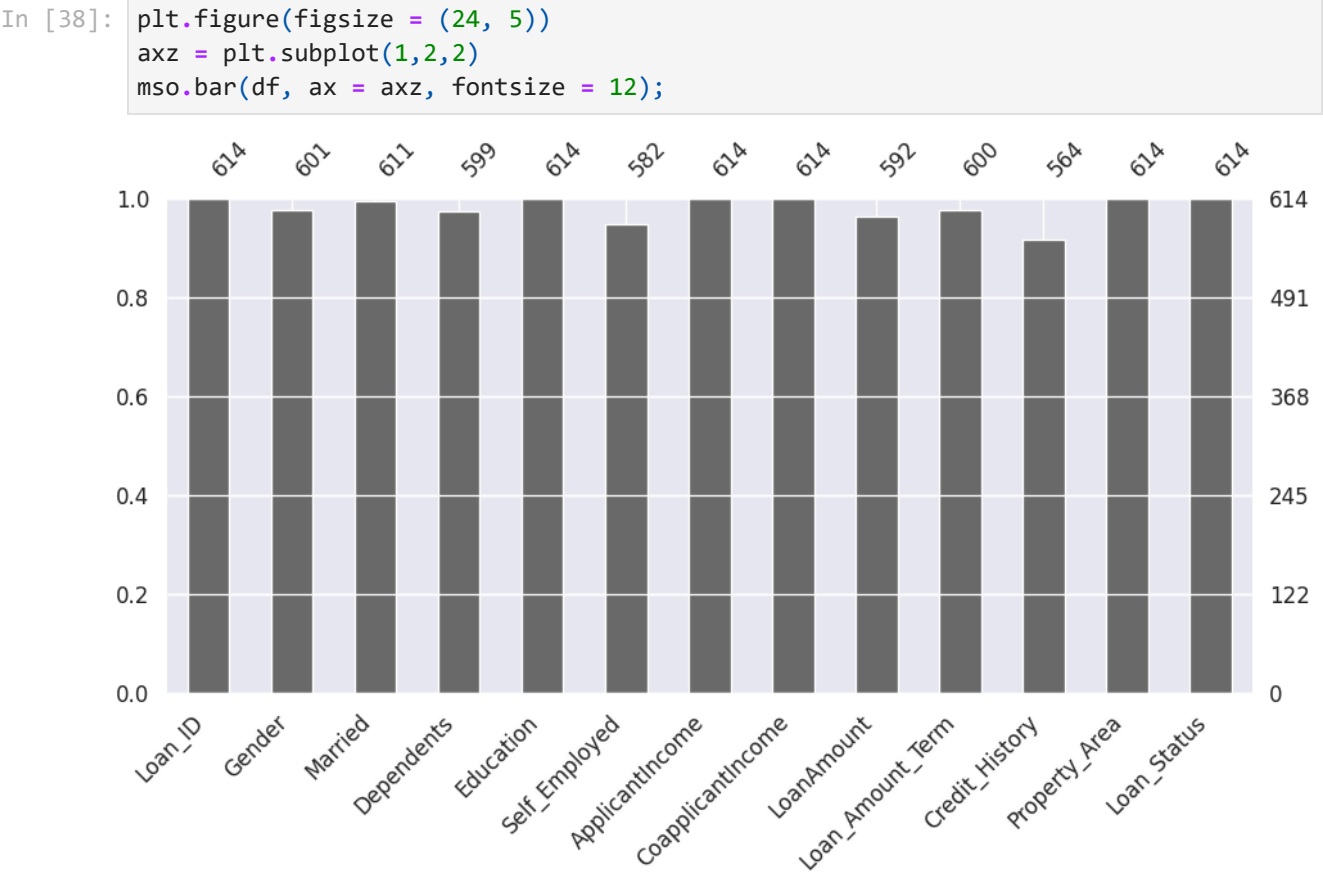
- Null values

```
In [37]: df.isnull().sum()
```

Out[37]:

	0
Loan_ID	0
Gender	13
Married	3
Dependents	15
Education	0
Self_Employed	32
ApplicantIncome	0
CoapplicantIncome	0
LoanAmount	22
Loan_Amount_Term	14
Credit_History	50
Property_Area	0
Loan_Status	0

dtype: int64



Data preprocessing

```
In [39]: df = df.drop(['Loan_ID'], axis = 1)
```

- Data imputation

```
In [40]: df['Gender'].fillna(df['Gender'].mode()[0],inplace=True)
df['Married'].fillna(df['Married'].mode()[0],inplace=True)
df['Dependents'].fillna(df['Dependents'].mode()[0],inplace=True)
df['Self_Employed'].fillna(df['Self_Employed'].mode()[0],inplace=True)
df['Credit_History'].fillna(df['Credit_History'].mode()[0],inplace=True)
df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mode()[0],inplace=True)

df['LoanAmount'].fillna(df['LoanAmount'].mean(),inplace=True)
df['ApplicantIncome'].fillna(df['ApplicantIncome'].mean(),inplace=True)
df['CoapplicantIncome'].fillna(df['CoapplicantIncome'].mean(),inplace=True)
```

/tmp/ipython-input-526222993.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Gender'].fillna(df['Gender'].mode()[0],inplace=True)
```

/tmp/ipython-input-526222993.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Married'].fillna(df['Married'].mode()[0],inplace=True)
```

/tmp/ipython-input-526222993.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Dependents'].fillna(df['Dependents'].mode()[0],inplace=True)
```

/tmp/ipython-input-526222993.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Self_Employed'].fillna(df['Self_Employed'].mode()[0],inplace=True)
```

/tmp/ipython-input-526222993.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Credit_History'].fillna(df['Credit_History'].mode()[0],inplace=True)
```

/tmp/ipython-input-526222993.py:6: FutureWarning: A value is trying to be set on a c

copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mode()[0],inplace=True)
/tmp/ipython-input-526222993.py:8: FutureWarning: A value is trying to be set on a c
opy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because
the intermediate object on which we are setting values always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['LoanAmount'].fillna(df['LoanAmount'].mean(),inplace=True)
/tmp/ipython-input-526222993.py:9: FutureWarning: A value is trying to be set on a c
opy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because
the intermediate object on which we are setting values always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['ApplicantIncome'].fillna(df['ApplicantIncome'].mean(),inplace=True)
/tmp/ipython-input-526222993.py:10: FutureWarning: A value is trying to be set on a
copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because
the intermediate object on which we are setting values always behaves as a copy.
```

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
df['CoapplicantIncome'].fillna(df['CoapplicantIncome'].mean(),inplace=True)
```

- One-hot encoding

```
In [41]: df = pd.get_dummies(df)

# Drop columns
df = df.drop(['Gender_Female', 'Married_No', 'Education_Not Graduate',
             'Self_Employed_No', 'Loan_Status_N'], axis = 1)

# Rename columns name
new = {'Gender_Male': 'Gender', 'Married_Yes': 'Married',
       'Education_Graduate': 'Education', 'Self_Employed_Yes': 'Self_Employed',
```



```
    'Loan_Status_Y': 'Loan_Status'}

df.rename(columns=new, inplace=True)
```

```
In [42]: print(df.shape)
```

```
(614, 17)
```

- Remove outliers

```
In [43]: num_df = df.select_dtypes(include='number')

Q1 = num_df.quantile(0.25)
Q3 = num_df.quantile(0.75)
IQR = Q3 - Q1

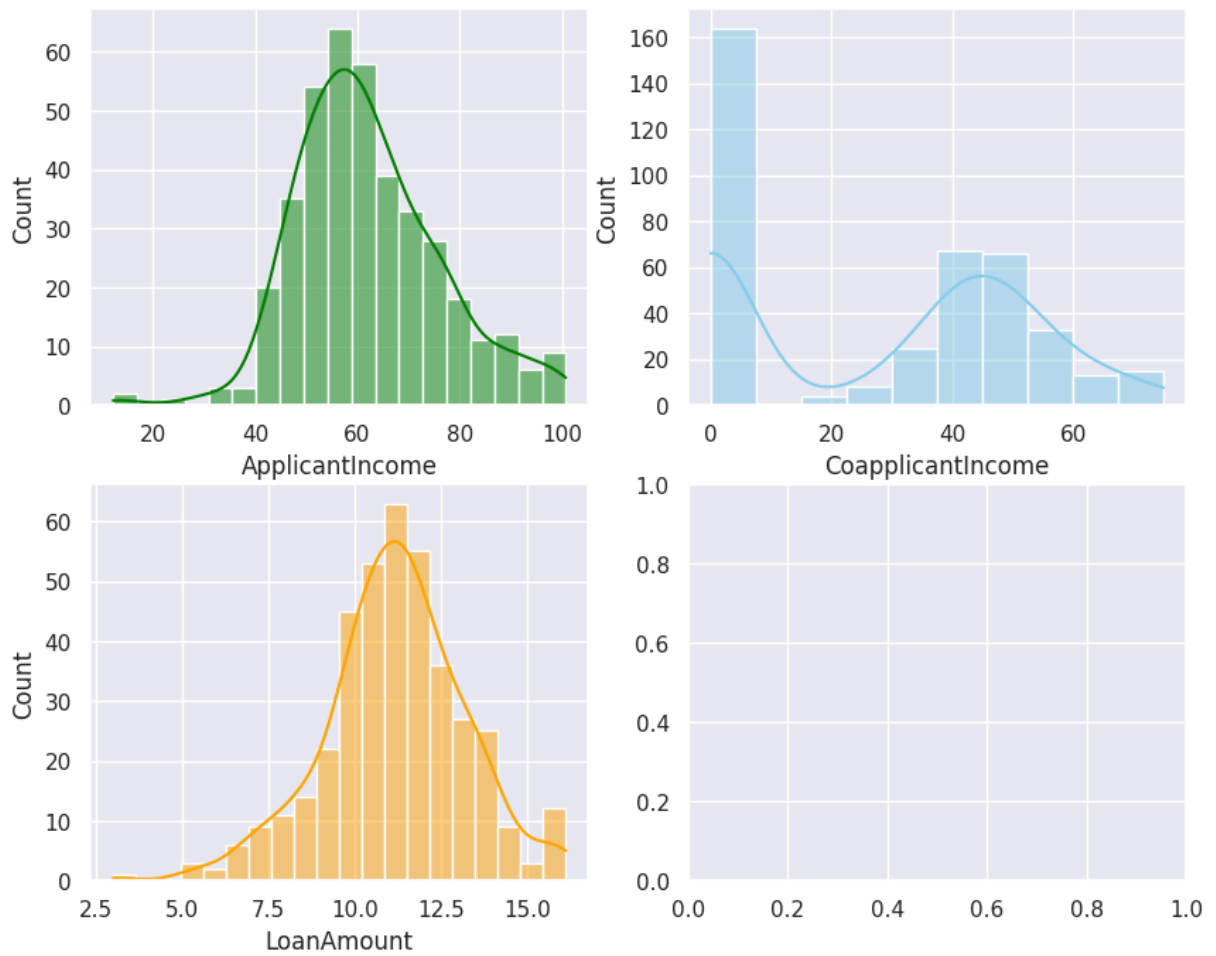
df = df[~((num_df < (Q1 - 1.5 * IQR)) | (num_df > (Q3 + 1.5 * IQR))).any(axis=1)]
```

- Skewed distribution treatment

```
In [44]: df.ApplicantIncome = np.sqrt(df.ApplicantIncome)
df.CoapplicantIncome = np.sqrt(df.CoapplicantIncome)
df.LoanAmount = np.sqrt(df.LoanAmount)
```

```
In [45]: sns.set(style="darkgrid")
fig, axs = plt.subplots(2, 2, figsize=(10, 8))

sns.histplot(data=df, x="ApplicantIncome", kde=True, ax=axs[0, 0], color='green')
sns.histplot(data=df, x="CoapplicantIncome", kde=True, ax=axs[0, 1], color='skyblue')
sns.histplot(data=df, x="LoanAmount", kde=True, ax=axs[1, 0], color='orange');
```



- Feature separation

```
In [46]: X = df.drop(["Loan_Status"], axis=1)
         y = df["Loan_Status"]
```

- SMOTE technique

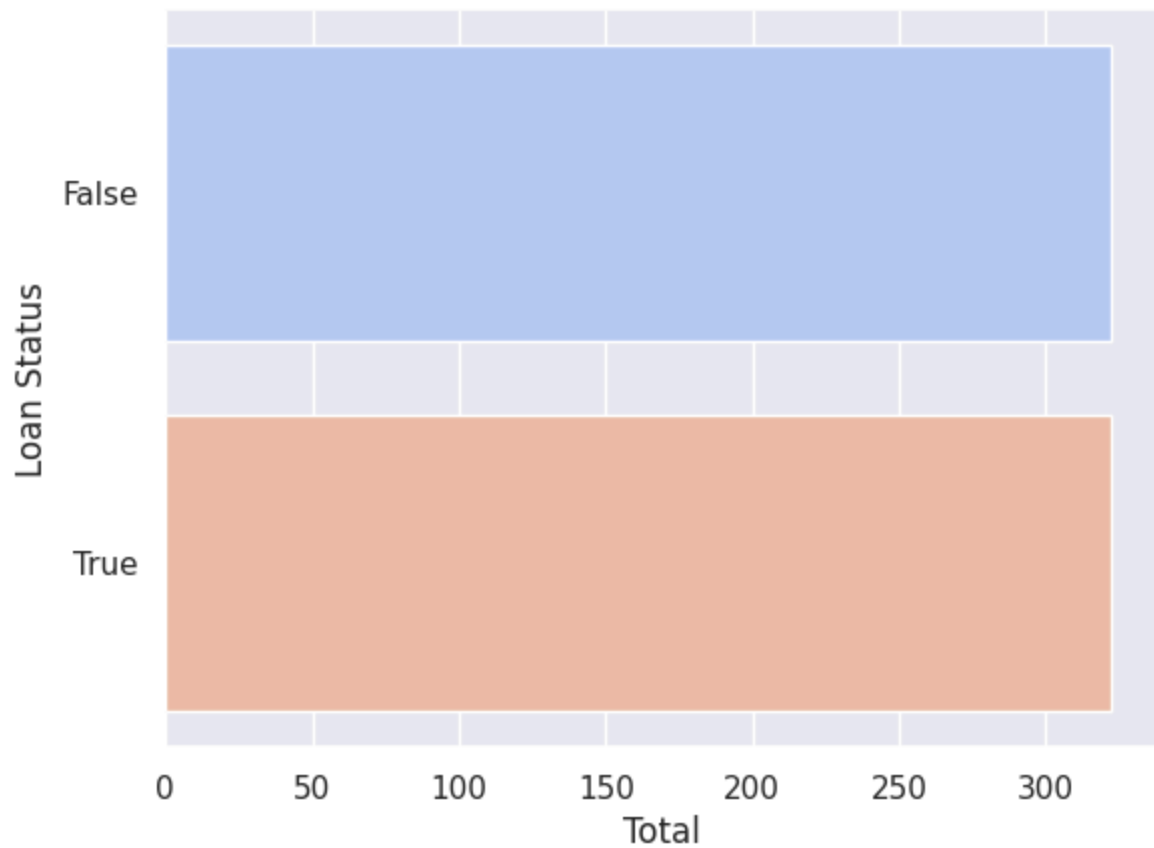
```
In [47]: X, y = SMOTE().fit_resample(X, y)
```

```
In [48]: sns.set_theme(style="darkgrid")
         sns.countplot(y=y, data=df, palette="coolwarm")
         plt.ylabel('Loan Status')
         plt.xlabel('Total')
         plt.show()
```

/tmp/ipython-input-537720374.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(y=y, data=df, palette="coolwarm")
```



- Data normalization

```
In [49]: X = MinMaxScaler().fit_transform(X)
```

- Splitting dataset

```
In [50]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_s
```

Models

- Logistic Regression

```
In [51]: LRclassifier = LogisticRegression(solver='saga', max_iter=500, random_state=1)
LRclassifier.fit(X_train, y_train)

y_pred = LRclassifier.predict(X_test)

print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))

from sklearn.metrics import accuracy_score
LRacc = accuracy_score(y_pred, y_test)
print('LR accuracy: {:.2f}%'.format(LRacc*100))
```

	precision	recall	f1-score	support
False	0.90	0.70	0.79	67
True	0.74	0.92	0.82	62
accuracy			0.81	129
macro avg	0.82	0.81	0.81	129
weighted avg	0.83	0.81	0.80	129

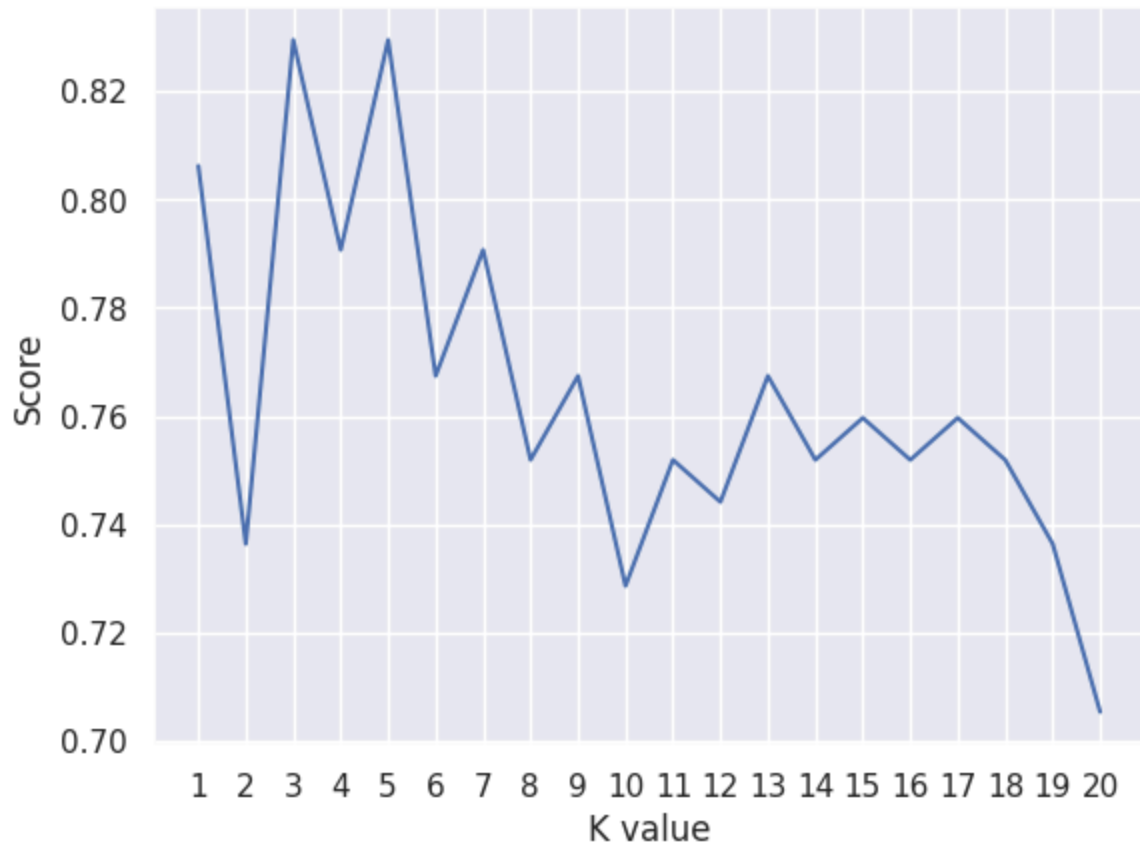
```
[[47 20]
 [ 5 57]]
```

LR accuracy: 80.62%

- K-Nearest Neighbour (KNN)

```
In [52]: scoreListknn = []
for i in range(1,21):
    KNclassifier = KNeighborsClassifier(n_neighbors = i)
    KNclassifier.fit(X_train, y_train)
    scoreListknn.append(KNclassifier.score(X_test, y_test))

plt.plot(range(1,21), scoreListknn)
plt.xticks(np.arange(1,21,1))
plt.xlabel("K value")
plt.ylabel("Score")
plt.show()
KNacc = max(scoreListknn)
print("KNN best accuracy: {:.2f}%".format(KNacc*100))
```



KNN best accuracy: 82.95%

- Support Vector Machine (SVM)

```
In [53]: SVCclassifier = SVC(kernel='rbf', max_iter=500)
SVCclassifier.fit(X_train, y_train)

y_pred = SVCclassifier.predict(X_test)

print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))

from sklearn.metrics import accuracy_score
SVCacc = accuracy_score(y_pred, y_test)
print('SVC accuracy: {:.2f}%'.format(SVCacc*100))
```

	precision	recall	f1-score	support
False	0.94	0.72	0.81	67
True	0.76	0.95	0.84	62
accuracy			0.83	129
macro avg	0.85	0.83	0.83	129
weighted avg	0.85	0.83	0.83	129

```
[[48 19]
 [ 3 59]]
SVC accuracy: 82.95%
```

- Naive Bayes

```
In [54]: NBclassifier2 = GaussianNB()
NBclassifier2.fit(X_train, y_train)

y_pred = NBclassifier2.predict(X_test)

print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))

from sklearn.metrics import accuracy_score
NBacc2 = accuracy_score(y_pred, y_test)
print('Gaussian Naive Bayes accuracy: {:.2f}%'.format(NBacc2*100))
```

	precision	recall	f1-score	support
False	0.68	0.66	0.67	67
True	0.64	0.66	0.65	62
accuracy			0.66	129
macro avg	0.66	0.66	0.66	129
weighted avg	0.66	0.66	0.66	129

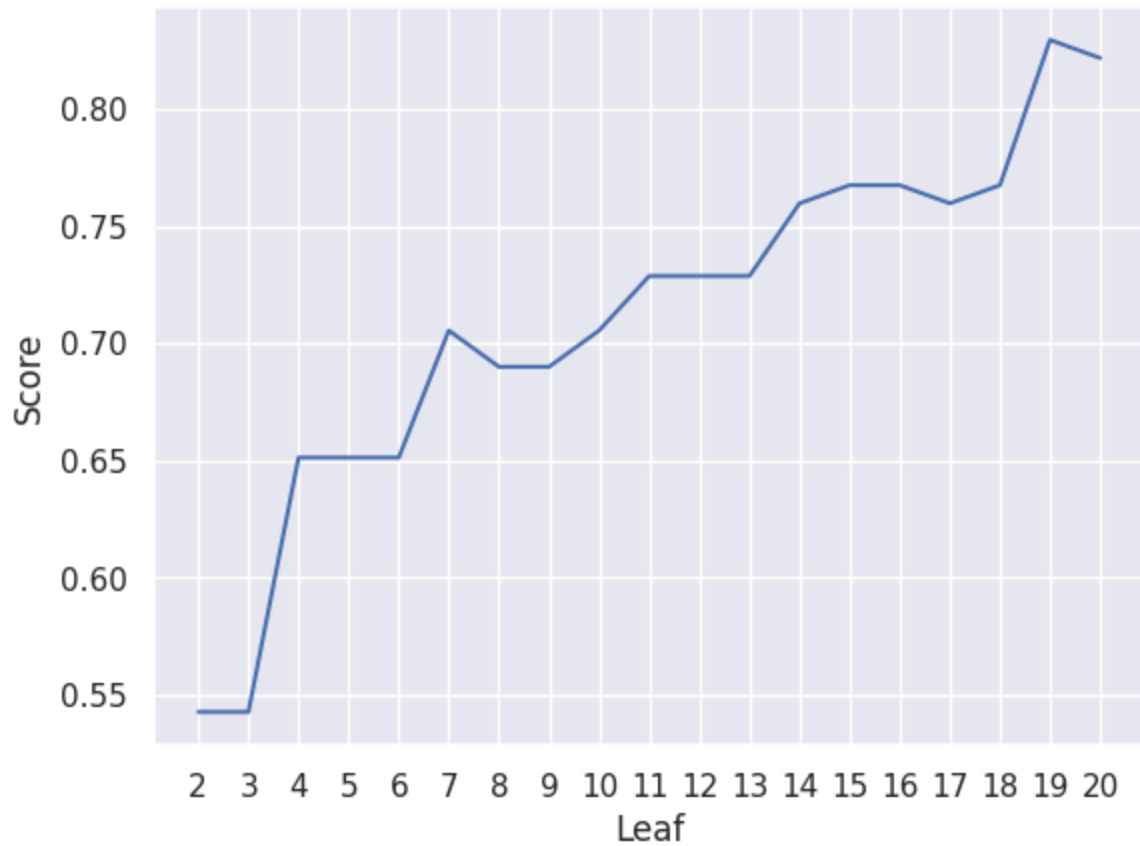
```
[[44 23]
 [21 41]]
```

Gaussian Naive Bayes accuracy: 65.89%

- Decision Tree

```
In [55]: scoreListDT = []
for i in range(2,21):
    DTclassifier = DecisionTreeClassifier(max_leaf_nodes=i)
    DTclassifier.fit(X_train, y_train)
    scoreListDT.append(DTclassifier.score(X_test, y_test))

plt.plot(range(2,21), scoreListDT)
plt.xticks(np.arange(2,21,1))
plt.xlabel("Leaf")
plt.ylabel("Score")
plt.show()
DTAcc = max(scoreListDT)
print("Decision Tree Accuracy: {:.2f}%".format(DTAcc*100))
```

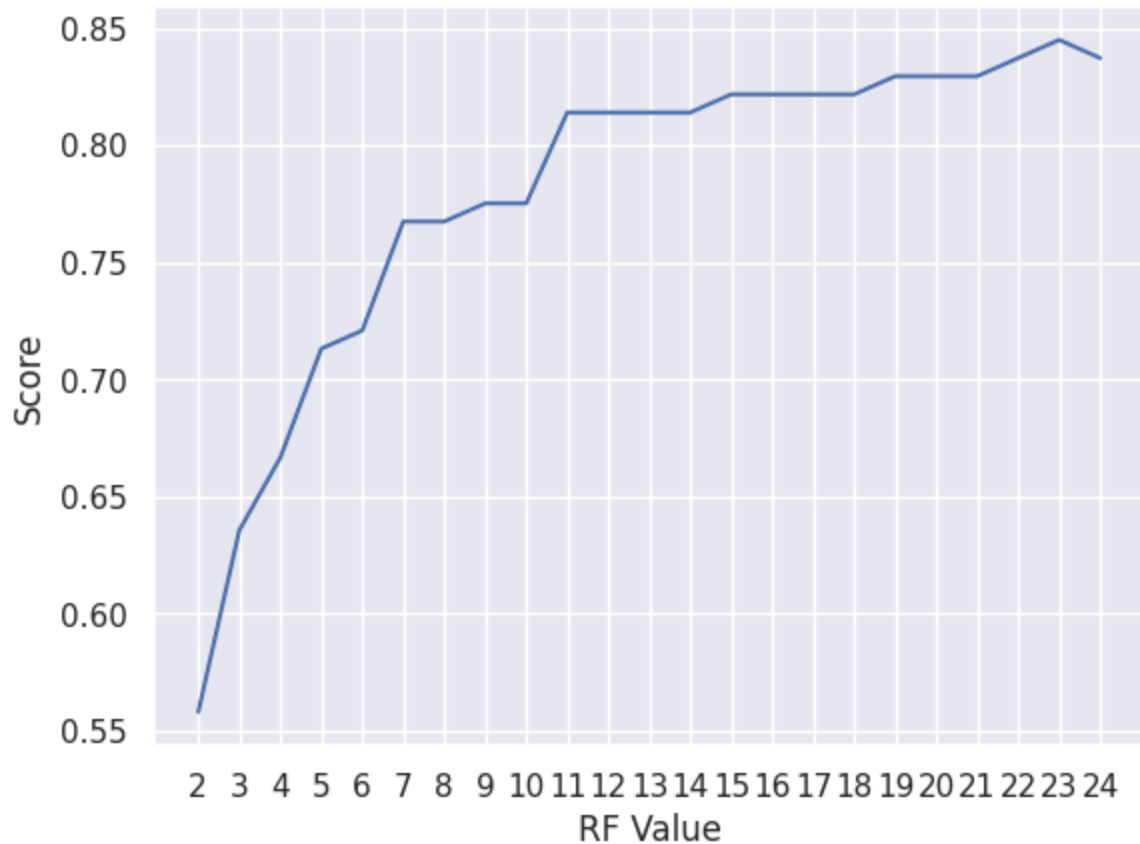


Decision Tree Accuracy: 82.95%

- Random Forest

```
In [56]: scoreListRF = []
for i in range(2,25):
    RFclassifier = RandomForestClassifier(n_estimators = 1000, random_state = 1, ma
    RFclassifier.fit(X_train, y_train)
    scoreListRF.append(RFclassifier.score(X_test, y_test))

plt.plot(range(2,25), scoreListRF)
plt.xticks(np.arange(2,25,1))
plt.xlabel("RF Value")
plt.ylabel("Score")
plt.show()
RFAcc = max(scoreListRF)
print("Random Forest Accuracy: {:.2f}%".format(RFAcc*100))
```



Random Forest Accuracy: 84.50%

- Gradient Boosting

```
In [57]: paramsGB={'n_estimators':[100,200,300,400,500],
                  'max_depth':[1,2,3,4,5],
                  'subsample':[0.5,1],
                  'max_leaf_nodes':[2,5,10,20,30,40,50]}
```

```
In [58]: GB = RandomizedSearchCV(GradientBoostingClassifier(), paramsGB, cv=20)
         GB.fit(X_train, y_train)
```

```
Out[58]: RandomizedSearchCV
          |
          | best_estimator_:
          | GradientBoostingClassifier
          |
          | GradientBoostingClassifier
```

```
In [59]: print(GB.best_estimator_)
         print(GB.best_score_)
         print(GB.best_params_)
         print(GB.best_index_)
```



```
GradientBoostingClassifier(max_depth=5, max_leaf_nodes=50, n_estimators=400,
                           subsample=1)
0.8384615384615387
{'subsample': 1, 'n_estimators': 400, 'max_leaf_nodes': 50, 'max_depth': 5}
6
```

```
In [60]: GBclassifier = GradientBoostingClassifier(subsample=0.5, n_estimators=400, max_depth=5)
GBclassifier.fit(X_train, y_train)

y_pred = GBclassifier.predict(X_test)

print(classification_report(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))

from sklearn.metrics import accuracy_score
GBAcc = accuracy_score(y_pred, y_test)
print('Gradient Boosting accuracy: {:.2f}%'.format(GBAcc*100))
```

	precision	recall	f1-score	support
False	0.79	0.81	0.80	67
True	0.79	0.77	0.78	62
accuracy			0.79	129
macro avg	0.79	0.79	0.79	129
weighted avg	0.79	0.79	0.79	129

```
[[54 13]
 [14 48]]
Gradient Boosting accuracy: 79.07%
```

Model comparison

```
In [61]: compare = pd.DataFrame({'Model': ['Logistic Regression', 'K Neighbors',
                                           'SVM', 'Gaussian NB', 'Decision Tree',
                                           'Random Forest', 'Gradient Boost'],
                                'Accuracy': [LRAcc*100, KNAcc*100, SVCAcc*100, NBAcc2*100,
                                              RFAcc*100, GBAcc*100]})
compare.sort_values(by='Accuracy', ascending=False)
```

Out[61]:

	Model	Accuracy
5	Random Forest	84.496124
2	SVM	82.945736
1	K Neighbors	82.945736
4	Decision Tree	82.945736
0	Logistic Regression	80.620155
6	Gradient Boost	79.069767
3	Gaussian NB	65.891473

Documentation

Summary

In this project we are classifying people as being eligible or not for a loan.

The dataset we are using has 13 variables: 1 id, 8 categorical and 4 numerical variables. We are trying to classify on the Loan Status variable.

We are also trying to determine the best model for this task, so we will be using 7 different models which will be compared in terms of accuracy.

Implementation

First thing was to explore the data from the dataset to better understand what we are working with.

During the exploration there are multiple variables which are unbalanced (ex. Gender, Credit History or even Loan Status).

Next, we explore correlations between the variables. We can see a positive correlation between Loan Amount and Applicant Income.

It is important to also determine the amount of outliers for each variable. We can see that for Applicant Income, Coapplicant Income and Loan Amount there are a considerable number of outliers.

Next comes the data preprocessing. First is the data imputation, which substitutes an estimated value for missing values in the dataset.

At this point we are also removing the outliers.

Last notable thing to do is to balance the dataset. As mentioned before, the Loan Status variable, which is used in our classification, is unbalanced. To fix that, we are using the SMOTE technique so that Loan Status can be balanced. This is done by artificially inserting entries in the dataset with estimated values based on other entries. This is important to avoid bias in the models.

Lastly, the data is normalized and split in training and testing sets.

Results

The models used, with their accuracies are the following:

- Logistic Regression - 80.62%
- KNN - 82.95%
- SVM - 82.95%
- Gaussian Naive Bayes - 65.89%
- Decision Tree - 82.95%
- Random Forest - 84.50%
- Gradient Boost - 79.07%

As a result, the best performing model is Random Forest, achieving 84.5%, closely followed by 3 models achieving the same accuracy of 82.95% (KNN, SVM, Decision Tree).

The worst performing model is the Gaussian Naive Bayes, achieving only 65.89% accuracy.